

ACADEMIC RESEARCH CASE STUDY • EMPIRICAL CODE STYLOMETRICS

Zero-Shot LLM Code Synthesis vs. Production-Hardened Human Reference Code: A Stylometric and Structural Case Study

Hassan Elkady — Computer Engineering Student, Arab Academy for Science, Technology and Maritime Transport (AAST)

August 2026 | Primary Frontier Dataset: N=76 OpenRouter LLM Runs vs n=10 Human Baseline | Auxiliary Dataset: N=50 Pre-Gen Recreations

Abstract

The rapid adoption of Large Language Models (LLMs) for automated code synthesis has sparked interest in evaluating the structural and qualitative properties of machine-generated code. However, existing benchmarks often focus strictly on functional pass rates (e.g., HumanEval, MBPP) rather than stylometric, performance, and maintenance characteristics. This paper presents an empirical case study evaluating **76 zero-shot synthetic code generations** produced by three frontier LLM architectures (*Google Gemini 3.5 Flash*, *OpenAI GPT-5.6 Sol*, and *Anthropic Claude Sonnet 4.6*) against a reference baseline of 10 production-hardened standard library functions authored by prominent human engineers prior to the LLM era (2017–2018 code from React 16, Go 1.10, Redis 5.0, Linux Kernel 4.14, Rust stdlib, PyTorch 1.0, and FastHTTP). In addition, we evaluate an auxiliary secondary benchmark of 50 pre-generated AI recreations.

Our quantitative analysis reveals statistically significant stylometric divergence: frontier synthetic implementations exhibit **+297% lines of code (LOC) expansion** (Mean = 59.62 ± 27.67 LOC vs. 15.00 ± 6.78 LOC human, Mann-Whitney U = 22.5, Holm-Bonferroni $p_{\text{adj}} = 9.04 \times 10^{-6}$, rank-biserial effect size $r_{\text{rb}} = +0.941$), elevated comment density ($13.73\% \pm 11.47\%$ synthetic vs. $1.43\% \pm 4.52\%$ human, $p_{\text{adj}} = 0.0019$, $r_{\text{rb}} = +0.686$), higher explicit type annotation density (13.13 ± 11.72 vs 1.50 ± 1.35 , $p_{\text{adj}} = 0.0004$, $r_{\text{rb}} = +0.774$), and higher vertical whitespace ratios ($17.47\% \pm 4.88\%$ synthetic vs. $5.54\% \pm 6.95\%$ human, $p_{\text{adj}} = 0.0002$, $r_{\text{rb}} = +0.812$). All confidence intervals are computed via non-parametric 10,000-resample bootstrapping.

Inter-model evaluations using Kruskal-Wallis H-tests demonstrate significant variance across LLM providers, highlighting Gemini 3.5 Flash as hyper-pedagogical (21.31% comment density), GPT-5.6 Sol as minimal (1.08% comment density), and Claude Sonnet 4.6 as highly verbose (73.53 LOC avg). We catalogue occurrence rates for seven structural patterns and document threats to validity, establishing that zero-shot LLM output reflects pedagogical abstraction rather than production-hardened human engineering idioms.

1. Introduction

Automated code generation powered by Large Language Models (LLMs) has transitioned from snippet autocompletion to end-to-end function synthesis. While LLMs achieve high pass rates on standard coding benchmarks, code quality in production software depends heavily on non-functional dimensions: maintainability, memory alignment, cache locality, control flow clarity, and domain-specific state invariant enforcement.

2. Related Work

Recent research has increasingly examined the quality, verbosity, and correctness of LLM-generated code: Binkley et al. (2023) and Jesse et al. (2023) observed significant verbosity in LLM code; Kabir et al. (2023) analyzed tutorial explanation habits in LLM outputs; Ugare et al. (2024) systematically categorized performance bugs in LLM code; and Nguyen et al. (2023) documented security and domain invariant omissions in Copilot outputs.

3. Methodology & Dataset Composition

Pre-AI Human Baseline Reference Dataset (n=10): Extracted from 2017–2018 repositories: React 16 Fiber Scheduler (Andrew Clark / Dan Abramov), React 16 shallowEqual, Go 1.10 strings.Builder (Russ Cox / Brad Fitzpatrick), Rust slice::rotate_left, PyTorch 1.0 clamp_out, Redis 5.0 raxInsert (antirez), TypeScript 3.0 isIdentifierStart (Anders Hejlsberg), Linux Kernel 4.14 eBPF htab_map_lookup_elem, FastHTTP caseInsensitiveCompare, and Rust Vec::retain.

Sampling Protocol & Variance Explanation: Single-shot stateless generation (temperature = 0.0 / default minimum variance), max_tokens = 750-800. System prompt: "Write clean, production-quality code. Output only the

code, no explanation." Reported LOC SD (± 27.67) reflects inter-task variance across 14 diverse algorithmic problems and inter-model variance across 3 model families, rather than intra-run stochastic noise.

Dataset Breakdown & Reconciliation (N=136 Total): Human Reference Baseline Records (n=10) + Primary Frontier LLM Dataset (N=76 OpenRouter runs) + Secondary Pre-Generated AI Recreations (N=50) = 136 Master Records Total.

4. Quantitative Stylometric Results & Statistical Significance

4.1 Primary Frontier Model Study: Human Reference (n=10) vs. Frontier LLMs (N=76)

Stylometric Metric	Human Reference (n=10)	Frontier LLMs (N=76)	Mann-Whitney U	Raw p-val	Holm-Bonferroni p_adj	Rank-Biserial Effect Size (r_rb)	FWER Significance
Lines of Code (LOC)	15.00 $\pm$ 6.78 [11.1, 18.9]	59.62 $\pm$ 27.67 [53.5, 65.9]	22.5	p = 1.51e-06	p_adj = 9.04e-06	r_rb = +0.941 (Massive)	Significant (p < 0.01)
Comment Density (%)	1.43% $\pm$ 4.52% [0.0, 4.3]	13.73% $\pm$ 11.47% [11.2, 16.3]	119.5	p = 3.73e-04	p_adj = 0.0019	r_rb = +0.686 (Large)	Significant (p < 0.01)
Explicit Type Annotations	1.50 $\pm$ 1.35 [0.7, 2.3]	13.13 $\pm$ 11.72 [10.6, 15.8]	86.0	p = 7.47e-05	p_adj = 0.0004	r_rb = +0.774 (Large)	Significant (p < 0.01)
Helper Method Count	0.00 $\pm$ 0.00 [0.0, 0.0]	1.25 $\pm$ 2.46 [0.7, 1.8]	260.0	p = 0.0416	p_adj = 0.0832	r_rb = +0.316 (Medium)	Marginally Significant
Return Statement Count	1.70 $\pm$ 1.49 [0.9, 2.6]	3.16 $\pm$ 2.31 [2.7, 3.7]	219.0	p = 0.0272	p_adj = 0.0816	r_rb = +0.424 (Med-Large)	Marginally Significant
Vertical Whitespace (%)	5.54% $\pm$ 6.95% [1.8, 9.8]	17.47% $\pm$ 4.88% [16.4, 18.5]	71.5	p = 3.32e-05	p_adj = 0.0002	r_rb = +0.812 (Massive)	Significant (p < 0.01)

4.2 Per-Model Breakdown & Kruskal-Wallis Inter-Model Significance Tests

Model Sub-Group	Record Count (N)	Mean LOC ($\pm$ SD)	Mean Comment Density (%)	Mean Type Annotations	Mean Helper Methods
Google Gemini 3.5 Flash	N=38	55.39 $\pm$ 23.81	21.31% $\pm$ 7.77%	10.97 $\pm$ 8.21	1.08 $\pm$ 1.42
OpenAI GPT-5.6 Sol	N=19	54.16 $\pm$ 35.19	1.08% $\pm$ 2.18%	10.74 $\pm$ 9.53	1.26 $\pm$ 1.85
Anthropic Claude Sonnet 4.6	N=19	73.53 $\pm$ 19.76	11.23% $\pm$ 10.61%	19.84 $\pm$ 14.21	1.58 $\pm$ 2.14
Kruskal-Wallis H-test	—	H = 8.93, p = 0.0115	H = 43.50, p = 3.58e-10	H = 4.98, p = 0.0829	H = 0.92, p = 0.6316

4.3 Secondary Benchmark Study: Pre-Generated AI Recreations (N=50)

Separately from the frontier models, the secondary dataset of 50 pre-generated AI recreations (5 versions for each of the 10 human prompts) exhibited an average LOC of 13.14 ± 5.21 and comment density of $2.39\% \pm 6.21\%$. Because these recreations solved short 15-line prompt tasks, their structural metrics closely matched the brevity of the human reference implementations, demonstrating that prompt scope strongly influences synthetic LOC volume.

5. Empirical Frequency Analysis of 7 Structural Patterns

Pattern Identifier & Description	Observed Frequency (k / 126)	Percentage (%)
Pattern 1: Structural Micro-Fragmentation — Decomposing simple tasks into ≥ 2 helper functions or extra class wrappers.	69 / 126	54.8%
Pattern 2: Contextual Invariant Omission — Omitting domain safety checks (struct copy guards, null prototype handling, Object.is).	6 / 126	4.8%
Pattern 3: Trivial Syntax-Echo Comments — Writing comments that directly parrot line syntax (# Increment counter).	28 / 126	22.2%
Pattern 4: Functional Iterator Closures in Hot Loops — Using .every(), .map(), .forEach() in performance hot loops.	5 / 126	4.0%
Pattern 5: In-Loop Mutating Array Shifts — Regressing runtime complexity from $O(N)$ to $O(N^2)$ via vector removals in loops.	1 / 126	0.8%
Pattern 6: Asynchronous State & Timer Lifecycle Leaks — Omitting clearTimeout() or mutating subscriber lists live.*	0 / 126*	0.0%*
Pattern 7: Compiler Vectorization Obstacles — Using nested std::min(std::max(...)) calls that hinder SIMD auto-vectorization.	1 / 126	0.8%

*Note on Pattern 6: Pattern 6 was identified during qualitative domain analysis of asynchronous primitives, but scored 0/126 in the quantitative dataset because Tier 2 async tasks (task_12 Event Emitter and task_14 TTL Cache) were skipped when OpenRouter API calls reached the hard payment budget limit.

6. Model Stylometric Profiles

Google Gemini 3.5 Flash (N=38): Hyper-pedagogical profile, highest task comment density ($21.31\% \pm 7.77\%$), extensive JSDoc wrappers.

OpenAI GPT-5.6 Sol (N=19): Enterprise minimal profile, lowest comment density ($1.08\% \pm 2.18\%$), coarse-grained global mutex locks.

Anthropic Claude Sonnet 4.6 (N=19): Comprehensive test-inclusive profile, highest task line count (73.53 ± 19.76 LOC), embedded unit tests.

7. Threats to Validity

Construct Validity: Comparing zero-shot LLM output against production-hardened open-source code authored by senior engineers measures the gap between initial machine generation and hardened software, not an average human developer.

Internal Validity: LLMs were evaluated strictly in single-shot mode. Multi-turn developer interaction, compiler error feedback, and code review shrink this quality gap.

Statistical Validity: While $N=76$ primary frontier generations provide high statistical power, the human reference sample ($n=10$) is small and Gemini was allocated 38 runs vs. 19 for GPT/Claude. Non-parametric Mann-Whitney U tests, Holm-Bonferroni FWER corrections, Kruskal-Wallis inter-model tests, bootstrap 95% CIs, and rank-biserial effect sizes were applied to address sample size asymmetry.

8. Conclusion & References

This study establishes statistically significant stylometric differences between zero-shot LLM code and production-hardened human reference code, providing empirical baseline metrics and frequency counts for automated code synthesis research.

References: Binkley et al. (2023) TSE; Jesse et al. (2023) ISSTA; Kabir et al. (2023) EMSE; Nguyen et al. (2023) ICSE; Ugare et al. (2024) PACMPL.

Appendix A: Task Catalog & Human Reference Alignment Mapping Table

Task / Flow ID	Category	Language	Title / Task Prompt	Human Reference Source
flow_01	Benchmark Flow	JavaScript	React 16 Fiber Scheduler Min-Heap (push/siftUp)	Facebook React 16 (A. Clark / D. Abramov)
flow_02	Benchmark Flow	JavaScript	React 16 Shallow Property Comparator (shallowEqual)	Facebook React 16 (D. Abramov)
flow_03	Benchmark Flow	Go	Go 1.10 strings.Builder.WriteString	Go Standard Library (R. Cox / B. Fitzpatrick)
flow_04	Benchmark Flow	Rust	Rust slice::rotate_left	Rust Standard Library Core Team
flow_05	Benchmark Flow	C++	PyTorch 1.0 clamp_out Tensor Operator	PyTorch Core (A. Paszke / S. Chintala)
flow_06	Benchmark Flow	C	Redis 5.0 Radix Tree Insert (raxInsert)	Redis Core (S. Sanfilippo - antirez)
flow_07	Benchmark Flow	TypeScript	TypeScript 3.0 Scanner (isIdentifierStart)	Microsoft TypeScript (A. Hejlsberg)
flow_08	Benchmark Flow	C	Linux Kernel 4.14 eBPF Hashtable (htab_map_lookup_elem)	Linux Kernel (A. Starovoitov / D. Borkmann)
flow_09	Benchmark Flow	Go	FastHTTP caseInsensitiveCompare	FastHTTP Engine (A. Valialkin)
flow_10	Benchmark Flow	Rust	Rust Vec::retain In-Place Predicate Filtering	Rust Standard Library Core Team
task_01 .. task_14	Research Task	Py / JS	14 Algorithmic Tasks (LRU, Trie, Dijkstra, TokenBucket, etc.)	Benchmark Algorithmic Problems